

Public Safety Alerts Aggregator & Explorer

Course: Software Development 2-ITC308 / ITC309
Assessment item 1-Group Project : Project Transition
Report

Due Date: July 31, 2026

Author: Critical Signal
Institute: Charles Sturt University

Instructor: Rod Loveday

Ettore Vescio
tv00047@gmail.com
11799221

Thomas Smith
thomas1134smith@gmail.com
11787500

Christian Finocchiaro
cajfino@gmail.com
11854560

Pradip Pandey
pradippandey555@gmail.com
11831987

Subject Code	ITC308 / ITC309
Assignment	Project Transition Report
Team name	Critical Signal
Project Title	2026-16 –Public Safety Alerts Aggregator & Explorer
Submission date	July 31, 2026
Modified at	19/07/2026 13:00:00

Project Aim and Background

During the last session, the Critical Signal team selected the Public Safety Alerts Aggregator & Explorer project in response to the growing challenge of accessing timely and reliable public safety information. Across New South Wales, emergency alerts relating to bushfires, floods, severe weather, road incidents and other hazards are published by multiple government agencies using different websites, data formats and interfaces. While each agency provides accurate information, the lack of a unified platform makes it difficult for members of the public to quickly identify alerts relevant to their location, particularly during rapidly evolving emergency situations.

The project was undertaken in response to a client brief provided as part of the Software Development Project and focuses on improving access to publicly available emergency information rather than replacing existing government systems. During the planning and analysis phase, stakeholder needs were identified, system requirements were documented, and a technical solution was designed to aggregate official alert feeds into a single web-based application. The planning phase also established the project's scope, assumptions and technical constraints, including reliance on publicly available data sources, third-party mapping services and the semester timeframe for development.

The overall aim of the project is to develop a scalable web application capable of collecting, standardising and presenting public safety alerts from multiple government agencies through a single searchable and interactive interface. By consolidating trusted information into one platform, the proposed system aims to improve public awareness, reduce information fragmentation and provide a strong foundation for future enhancements such as personalised alert subscriptions, analytics and additional data sources.

Project Overview

Project Summary

Project Name	2026-16 –Public Safety Alerts Aggregator & Explorer
Domain	Emergency Management and Public Information Systems

Purpose

To provide a centralised web application that automatically collects, standardises and displays official public safety alerts from multiple government agencies, enabling users to quickly identify hazards relevant to their location and subscribe to personalised email notifications.

Problem Being Addressed

Public safety alerts are currently distributed across multiple government websites and services, including weather warnings, bushfire alerts, flood notices and traffic incidents. Although these sources provide reliable information, they use different interfaces, formats and update mechanisms, making it difficult for users to obtain a complete and timely view of hazards affecting their area. The project addresses this fragmentation by aggregating official alerts into a single web-based platform that presents consistent, searchable and map-based information.

Expected Benefits

- Centralises alerts from multiple trusted government agencies into a single platform.
- Improves public awareness through real-time, map-based visualisation of hazards.
- Reduces the time required for users to locate relevant emergency information.
- Provides a scalable architecture capable of supporting additional alert sources and future system enhancements.

Primary Users

- General Public
- Emergency Services
- Government Agencies
- Researchers

Key Artefacts Completed

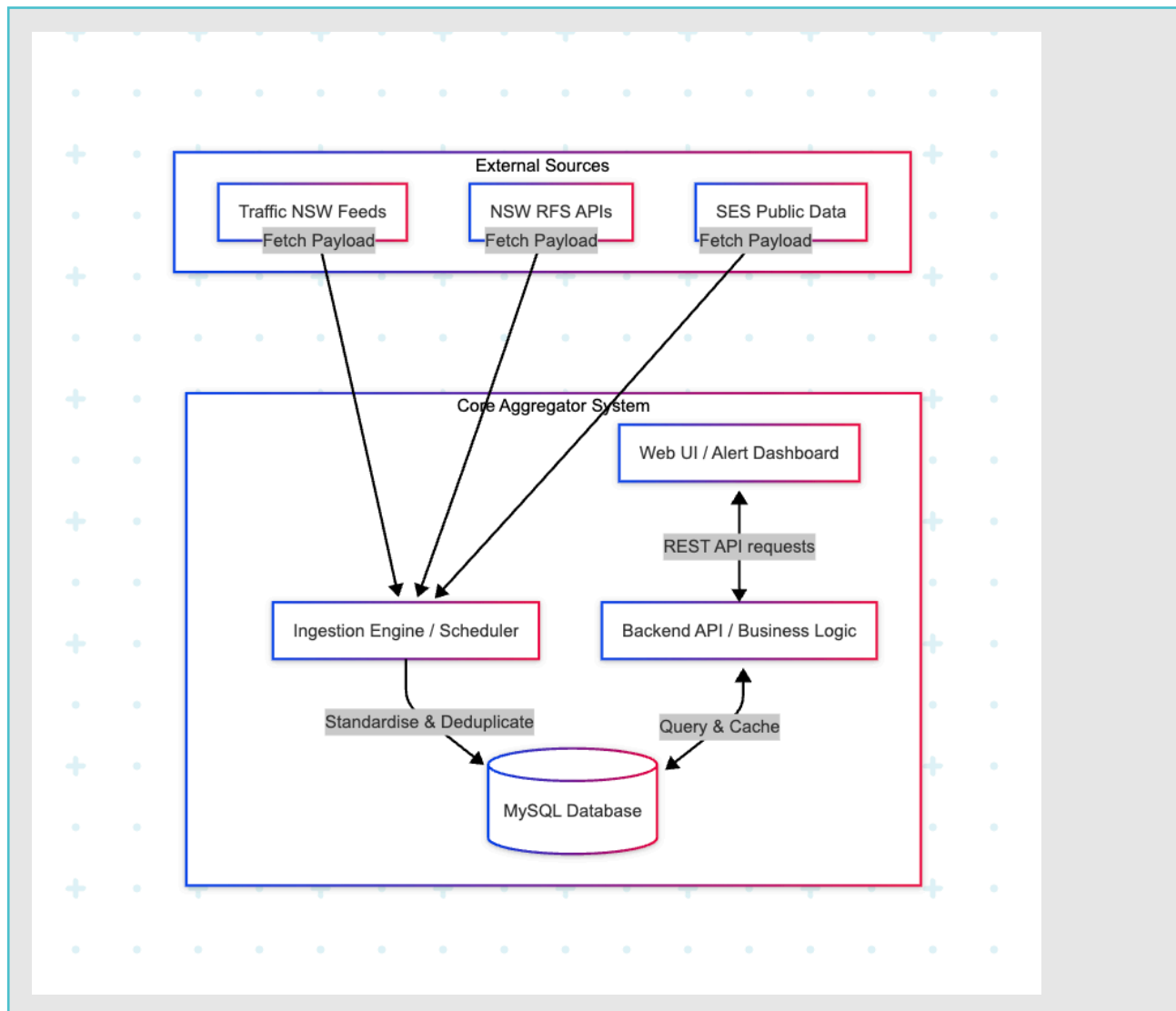
The planning and analysis phase produced a number of project artefacts that will guide implementation during Software Development 2. These artefacts establish the agreed project scope, document stakeholder requirements, define the proposed system architecture and provide a basis for implementation, testing and validation. All project artefacts are maintained within the team's GitHub repository and can be referenced throughout the implementation phase. <https://github.com/Azmith47/public-safety-alerts-aggregator.git>

Artefact	Purpose	Contribution to Development	Location
Client Brief	Defines client expectations and project objectives	Ensures implementation remains aligned with stakeholder needs	Client Brief
Project Charter	Defined project scope, objectives, stakeholders and project governance.	Provides the agreed project direction and implementation boundaries.	Project Charter
Requirements Baseline	Captured functional and non-functional requirements.	Primary reference for software implementation and testing.	Requirements Baseline
Stakeholder Analysis	Identified stakeholder needs and priorities.	Guides interface design and validation.	Requirements Baseline

Use Case Diagram	Defines interactions between users and the system.	Supports frontend and backend implementation.	Requirements Baseline
System Architecture	Defines the overall software architecture.	Guides backend development and deployment.	Requirements Baseline
Entity Relationship Diagram	Defines the relational database schema.	Used to implement the database.	Requirements Baseline
Wireframe Prototype	Illustrates the proposed interface.	Guides frontend implementation.	Requirements Baseline
Proofs of Concept	Validated ingestion, backend API and frontend approaches.	Reduces implementation risk by confirming technical feasibility.	Requirements Baseline

Key Design Artefacts

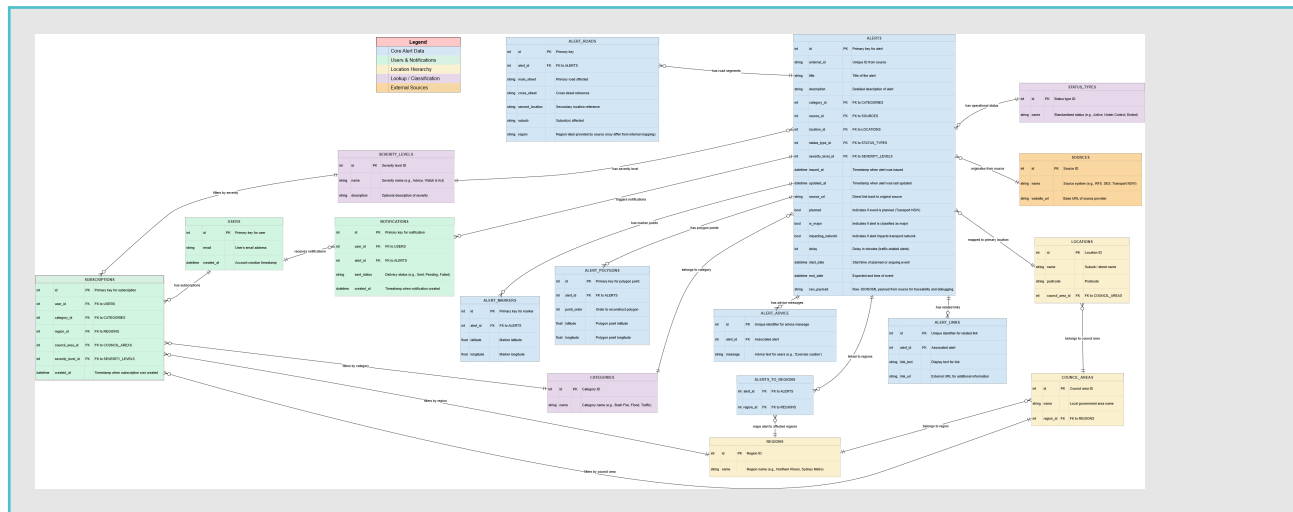
System Architecture



The system architecture illustrates the separation between the data ingestion layer, backend API, relational database and web interface. This design promotes

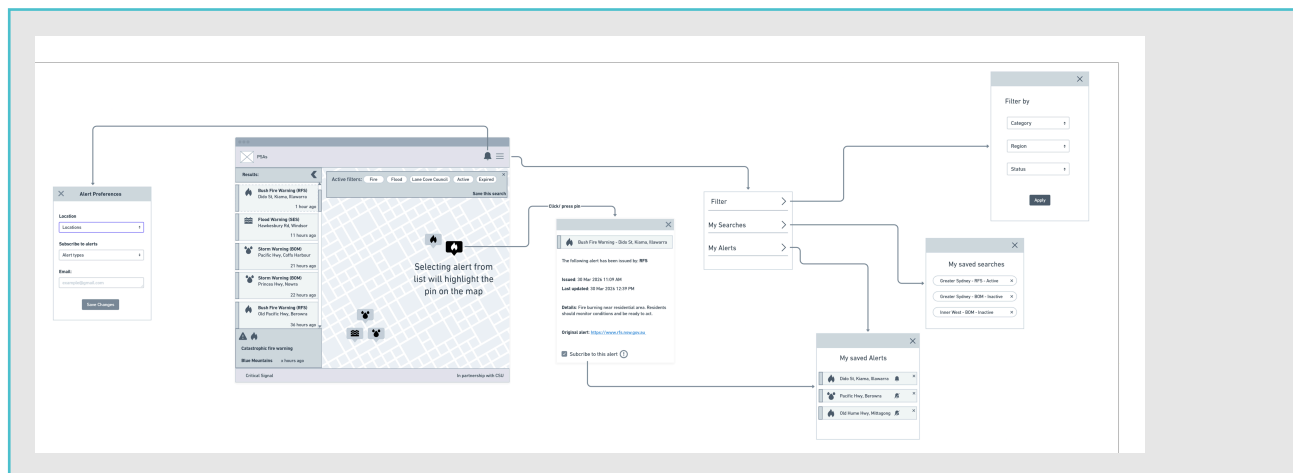
modularity by isolating source-specific ingestion logic from application logic, making it easier to extend the platform with additional alert providers.

Entity Relationship Diagram



The entity relationship diagram defines the relational database structure used to store standardised alert information, geographic regions, subscriptions and spatial data. It provides the blueprint for implementing the database schema during the development phase.

Wireframe



The wireframe demonstrates the intended user experience, including interactive map visualisation, alert filtering and subscription management. It provides a clear reference for frontend implementation while ensuring the interface remains aligned with stakeholder requirements identified during planning.

Team Members and Roles

Team Member	Planning	Development
Ettore Vescio	Project Manager & database design	Team Lead, backend development, database implementation
Christian Finocchiaro	UI & accessibility planning	Frontend
Thomas Smith	Requirements & Architecture	Backend
Pradip Pandey	QA planning	Testing & Security

Team Collaboration

The team intends to continue using an Agile approach throughout the implementation phase, with regular meetings, GitHub for version control and issue tracking, and Discord for day-to-day communication. Development tasks will be allocated according to each member's strengths while maintaining regular code reviews and collaborative testing to ensure consistent quality across the project. Project documentation produced during ITC303 will continue to be updated as implementation progresses, ensuring that design decisions and completed functionality remain well documented.

Development Outlook

The planning phase produced several artefacts and prototype components that are sufficiently mature to transition directly into implementation. While some refinement will occur during development, the project's architecture, database design and core interface have already been validated.

Ready for Implementation

- The overall frontend layout, including the navigation components, alert list and interactive map view, has been prototyped and validated during the planning phase. These components can be reused with only minor refinements during implementation.
- The project's CSS framework has been established using reusable variables and modular styling, allowing visual themes and layout adjustments to be made without extensive code changes.
- The alert card components have been designed and refined to present the most important incident information at a glance. Only minor adjustments are expected once live data is integrated.
- The data ingestion pipeline for fire and traffic alerts has been implemented and is ready for integration with the database layer. Additional alert sources can be incorporated by extending the existing normalisation framework.
- The interactive map component has been developed to display alert markers and polygons and is ready for integration with live alert data.
- The selected technology stack has already been configured with supporting development tools, including static application security testing (SAST), dependency scanning and application logging.
- The project's overall processing workflow — from collecting alerts, through data normalisation and storage, to presenting information within the web dashboard — has been defined and provides a solid foundation for implementation.
- The external government data sources have been identified and evaluated, allowing validation rules and data boundaries to be established before full implementation begins.

Needs Refinement

- Accessibility features, including ARIA labelling and screen reader compatibility, require comprehensive testing to ensure compliance across multiple assistive technologies.
- Responsive layouts for tablets and mobile devices require further refinement through additional media queries and usability testing.
- The interaction between the alert list, interactive map and detailed alert view requires final design decisions to ensure users can easily navigate between related information.
- The integration of additional alert sources, particularly weather alerts, requires further investigation to determine the most effective normalisation strategy.
- Database retention policies and long-term data persistence require refinement to balance historical information with storage efficiency.
- Performance optimisation techniques, including marker clustering and efficient rendering of spatial data, will be implemented to maintain responsiveness when displaying large numbers of alerts.
- Additional security controls, including Content Security Policy (CSP), HTTPS enforcement, rate limiting and secure secret management, require final implementation and validation.
- Privacy requirements, including data classification, retention periods and handling of personal information, require further documentation and review.
- The scope and depth of automated testing, integration testing, dynamic application security testing (DAST) and penetration testing require further planning.
- Monitoring and alerting requirements must be refined to determine which operational events should be logged, monitored and escalated during production.

Technical Challenges

- The project is being developed using React and Next.js, technologies that are relatively new to the team. This increases the likelihood of implementation issues and requires ongoing learning throughout development.
- The frontend, backend and database components have largely been developed independently during the planning phase and must now be successfully integrated into a single cohesive application.
- Integrating live alert data into the frontend while maintaining acceptable rendering performance will require careful state management and optimisation.
- The scheduled ingestion service must balance timely retrieval of new alerts with efficient resource usage, particularly when polling multiple external providers.
- Rendering large numbers of markers and polygons simultaneously presents performance challenges that will require optimisation techniques such as clustering and lazy rendering.
- External government data feeds cannot be fully controlled by the project team, requiring robust validation, sanitisation and error handling before data is processed.
- Displaying externally sourced alert information safely within the web interface requires protection against cross-site scripting (XSS) and other frontend injection attacks.
- The JavaScript ecosystem evolves rapidly, requiring ongoing monitoring of third-party dependencies for newly disclosed security vulnerabilities.
- Secure management of API keys, webhook secrets and other credentials across local development and deployment environments presents an ongoing operational challenge.
- Effective monitoring and logging must be implemented to distinguish normal operational failures from genuine security incidents while maintaining complete traceability across the ingestion pipeline, backend services and user interface.

Risks

- Implementing persistent user preferences and saved searches without user accounts introduces challenges around client-side storage, browser compatibility and long-term reliability.
- Integration with third-party email services introduces operational risks, including service outages, delivery failures and incorrect notification frequency, all of which require comprehensive testing.
- Poorly configured search filters may either exclude relevant alerts or overwhelm users with excessive information, reducing the overall effectiveness of the system.
- As additional data sources are introduced, database growth may negatively affect storage requirements and query performance if retention policies are not carefully managed.
- Insufficient input validation may allow malformed or malicious external data to propagate through the application.
- Delaying implementation of security controls until later development stages may increase project complexity and require significant rework.
- Limited system monitoring and observability may delay detection of service outages, malicious activity or unexpected application behaviour.
- Third-party software dependencies may introduce security vulnerabilities that require ongoing monitoring and timely updates.
- Incorrect configuration of the source code repository or continuous integration pipeline could expose sensitive credentials or weaken the project's overall security posture.

Development Priorities

1. Establish development environment
2. Implement database schema
3. Create backend API
4. Develop ingestion service
5. Develop frontend interface
6. Testing and deployment

How Documentation Supports Implementation

1. The Project Charter defines the agreed project scope and objectives.
2. The Requirements Baseline provides the functional and non-functional requirements that will guide implementation.
3. The System Architecture diagram supports component development and integration.
4. The Entity Relationship Diagram provides the blueprint for implementing the database schema.
5. The Wireframe Prototype guides frontend development and usability testing.
6. Acceptance criteria established during planning will form the basis of system validation and demonstration.

Readiness for the Next Phase

Overall, the project is well positioned to transition from planning into implementation. The planning and analysis phase produced a comprehensive set of artefacts that clearly define the project scope, stakeholder expectations, functional requirements and technical architecture. These artefacts significantly reduce uncertainty during development and provide a solid foundation for implementation, testing and project management.

While some refinements will naturally occur as external data sources are integrated and the user interface is evaluated, the project's overall scope remains realistic for the available timeframe. The team possesses complementary technical skills covering backend development, frontend implementation, database design and testing, allowing work to be undertaken in parallel while maintaining shared responsibility for integration and quality assurance.

The first development priorities will be establishing the development environment, implementing the database schema and data ingestion service, followed by development of the backend API and frontend interface. With the planning phase complete, the project is considered ready to proceed into full software development.